

1. PROJECT OVERVIEW

- **Project Title:** ComicCraft - AI Comic Story Creator using Gemini Models

2. ARCHITECTURAL BLUEPRINT :

ComicCraft is constructed using a robust, highly decoupled three-layer architecture. This modular design separates the Client Interface, the Application Server, and the Cloud Intelligence layers, ensuring maximum scalability, rapid asynchronous processing, and easy maintainability.

3. CLIENT INTERFACE LAYER (FRONTEND)

- **Primary Technologies:** HTML5, CSS3, Jinja2 Template Engine.
- **Function:** Handles data collection and the final visual presentation to the user.
- **Key Components:**
 - `index.html`: The data entry form capturing the user's narrative and aesthetic parameters.
 - `preview.html`: A dynamic webpage that parses Python backend arrays to display the generated comic strip fluidly on the screen.

4. APPLICATION SERVER LAYER (BACKEND)

- **Primary Technologies:** Python 3.10, FastAPI Framework, Uvicorn ASGI Server.
- **Function:** The core processing hub. It receives frontend data, sequences the API calls, manages network exceptions, and formats the output.
- **Module Breakdown:**
 - `app_routes.py`: The main controller managing HTTP endpoints and the execution order of the AI generation.
 - `gemini_logic.py`: Handles secure HTTPS transmission to Google's APIs to fetch the story structure and dialogues.

- `image_synthesis.py`: Executes asynchronous calls to Hugging Face to download `.jpg` assets, featuring localized fallback scripts for network failures.
- `pdf_engine.py`: Employs the FPDF module to mathematically align the text strings and image paths, rendering the final downloadable document.

5. CLOUD INTELLIGENCE LAYER (APIs) :

The system relies on external cloud infrastructure for all generative heavy lifting, communicating via secure HTTPS protocols.

- **Narrative Logic:** Google Generative AI (Gemini 1.5 Flash and Gemini 1.5 Pro).
- **Visual Rendering:** Hugging Face Inference API (Stable Diffusion v1.5).
- **Credential Security:** All API tokens are dynamically loaded into the environment using `python-dotenv`, preventing hardcoded security vulnerabilities.

6. DATA & ASSET MANAGEMENT To maintain high performance and low server costs, the application operates in a stateless manner without a heavy SQL database.

- **Temporary Image Storage:** Downloaded panel illustrations are stored briefly in `static/panels/`.
- **Document Compilation:** The compiled PDF output is hosted temporarily in `static/exports/` for user retrieval.

7. SYSTEM EXECUTION PIPELINE

1. Form data is passed from Frontend to `app_routes.py`.
2. Backend requests story JSON from `gemini_logic.py`.
3. Backend requests character dialogue from `gemini_logic.py`.
4. Backend requests images from `image_synthesis.py`.
5. The `pdf_engine.py` aggregates data and generates the `.pdf`.

6. Frontend Jinja2 template is rendered with the final dataset.